

UNIVERSIDAD DE MONTERREY

Practica 2

Microcontroladores

Alejandro Reyes

Nombre: Mat:

Nombre: Ivan Alejandro Navarro Alonso Mat: 617640

Nombre: David Cabrera Mat: 649032

“Damos nuestra palabra de que hemos hecho esta actividad con integridad académica”

Monterrey, N. L. a 11 de junio del 2026

Introducción

Los microcontroladores PIC permiten controlar matrices de LEDs (displays de puntos) mediante técnicas de multiplexado. En esta práctica se desarrollaron dos programas en lenguaje C, compilados con XC8, que controlan una matriz de LEDs de 8x8 conectada a los puertos B (filas) y D (columnas) de un microcontrolador PIC. El primer programa muestra un patrón gráfico estático con efecto de barrido, mientras que el segundo despliega de forma cíclica cuatro letras (D, I, V, A) con refresco por filas.

El objetivo es comprender cómo el multiplexado por filas, el uso de arreglos de patrones de bits y la manipulación de máscaras permiten generar imágenes en una matriz LED con recursos mínimos de hardware.

Marco Teórico

Multiplexado en matrices LED:

Una matriz LED de 8x8 tiene 8 filas y 8 columnas. En lugar de controlar cada LED individualmente (lo que requeriría 64 pines), el multiplexado activa una fila a la vez y aplica el patrón de columnas correspondiente. Si el ciclo se repite lo suficientemente rápido (generalmente más de 60 Hz), la persistencia visual del ojo humano genera la ilusión de que toda la imagen está encendida simultáneamente.

Registros TRISB, TRISD, PORTB y PORTD:

PORTB controla las filas de la matriz y PORTD controla las columnas. Ambos se configuran como salida (TRIS = 0x00). En cada ciclo de refresco se activa una única fila mediante PORTB y se escribe el byte de columnas en PORTD. El desplazamiento de bits ($1 \ll i$) permite activar la fila i -ésima sin valores codificados a mano.

Inversión y espejo de bits:

La función `invertirBits()` realiza un espejo horizontal del byte mediante tres operaciones de desplazamiento y máscaras. Esto corrige la orientación de los caracteres cuando el cableado de la matriz es el inverso al orden lógico de los bits. La expresión `~invertirBits(patron[i])` adicionalmente invierte la lógica (activo en bajo vs activo en alto) según lo requiera el hardware.

Retardos por software:

La función `__delay_ms()` genera una pausa calculada a partir de `_XTAL_FREQ`. En ambos programas se declara 8 MHz y se usa oscilador HS, por lo que la frecuencia declarada es coherente con el hardware. Un retardo de 5 ms por fila en una matriz de 8 filas da un período de refresco de 40 ms (25 Hz), suficiente para percibir la imagen sin parpadeo notable a la velocidad de refresco efectiva.

Desarrollo

Código 1 — Patrón gráfico con arreglo fijo:

El primer programa define un arreglo de 8 bytes que codifica un patrón gráfico (`patron[8] = {0x81, 0x42, 0x24, 0x18, 0x18, 0x24, 0x42, 0x81}`). Dentro del ciclo principal, se itera sobre las 8 filas: se activa la fila i con ($1 \ll i$) en PORTB y se escribe el byte correspondiente del arreglo en

PORTD, con un retardo de 5 ms entre cada fila. El resultado visual es un patrón simétrico en forma de X o rombo, que parece estático debido al efecto de persistencia visual.

El oscilador se configura en modo HS con 8 MHz, lo que es coherente con la macro `_XTAL_FREQ` declarada. El Watchdog Timer y la programación de bajo voltaje están desactivados; el reset por bajo voltaje está habilitado.

Código 2 — Despliegue de letras DIVA:

El segundo programa define cuatro arreglos de 8 bytes que codifican las letras D, I, V y A en formato de mapa de bits para una matriz de 8x8. Una función auxiliar `invertirBits()` realiza el espejo horizontal de cada byte. La función `mostrarPatron()` recibe un puntero al arreglo y un número de repeticiones: activa fila por fila durante 5 ms cada una, aplicando la inversión de bits y de lógica (`~invertirBits(patron[i])`). Con 60 repeticiones por letra, cada carácter permanece visible aproximadamente 2.4 segundos antes de pasar al siguiente.

El ciclo principal llama a `mostrarPatron()` para cada letra en orden ($D \rightarrow I \rightarrow V \rightarrow A$) de forma continua, generando una animación de texto desfilante. La configuración de fuses es idéntica al primer programa.

Comparación entre ambos códigos:

Aspecto	Código 1	Código 2
Oscilador (FOSC)	HS	HS
Frecuencia declarada	8 MHz (coherente)	8 MHz (coherente)
Control de LEDs	Escritura directa de byte completo por fila	Inversión de bits y lógica negada por fila
Patrón visual	Figura fija simétrica (tipo rombo/X)	Letras D, I, V, A en secuencia animada
Función auxiliar	Ninguna	<code>invertirBits()</code> y <code>mostrarPatron()</code>

Conclusión

El desarrollo de esta práctica permitió comprobar que el control de una matriz LED mediante un microcontrolador PIC requiere comprender tres aspectos fundamentales: la técnica de multiplexado por filas, la representación de imágenes como arreglos de bytes en mapa de bits, y la manipulación de bits para adaptar la orientación y la lógica de activación al cableado físico del hardware.

El primer programa demostró que con un arreglo simple y multiplexado por barrido de filas es posible mostrar un patrón gráfico estático de manera eficiente. El segundo programa extendió este concepto al incorporar una función de espejo horizontal de bits y la capacidad de mostrar múltiples caracteres en secuencia, lo que introduce los fundamentos de las animaciones en pantallas matriciales.

En ambos casos, la coherencia entre el oscilador configurado (HS a 8 MHz) y la macro `_XTAL_FREQ` garantizó que los retardos de 5 ms por fila fueran precisos, resultando en un refresco visual estable. Estos ejercicios consolidan las bases necesarias para abordar displays más complejos y sistemas embebidos con mayor demanda gráfica.